OverYonder.ai
Platform Module Architecture

Technical specification of the six core modules: Tenants, Integrations, Tools, Skills, Use Cases, and Actions — how they are structured, how they connect, and how they drive the agentic execution pipeline.

Prepared for GPT UX collaboration session

March 2026

Table of Contents

- 1. Architecture Overview
- 2. Module 1: Tenants
- 3. Module 2: Integrations
- 4. Module 3: Tools
- 5. Module 4: Skills
- 6. Module 5: Use Cases
- 7. Module 6: Actions
- 8. Cross-Module Relationships
- 9. End-to-End Execution Flow
- 10. Data Model Summary

1. Architecture Overview

The OverYonder.ai platform is built on a modular architecture where six core modules form a dependency chain that drives agentic AI workflows:

```
Tenants → Integrations → Tools → Skills → Use Cases → Actions
```

Each module builds on the ones before it:

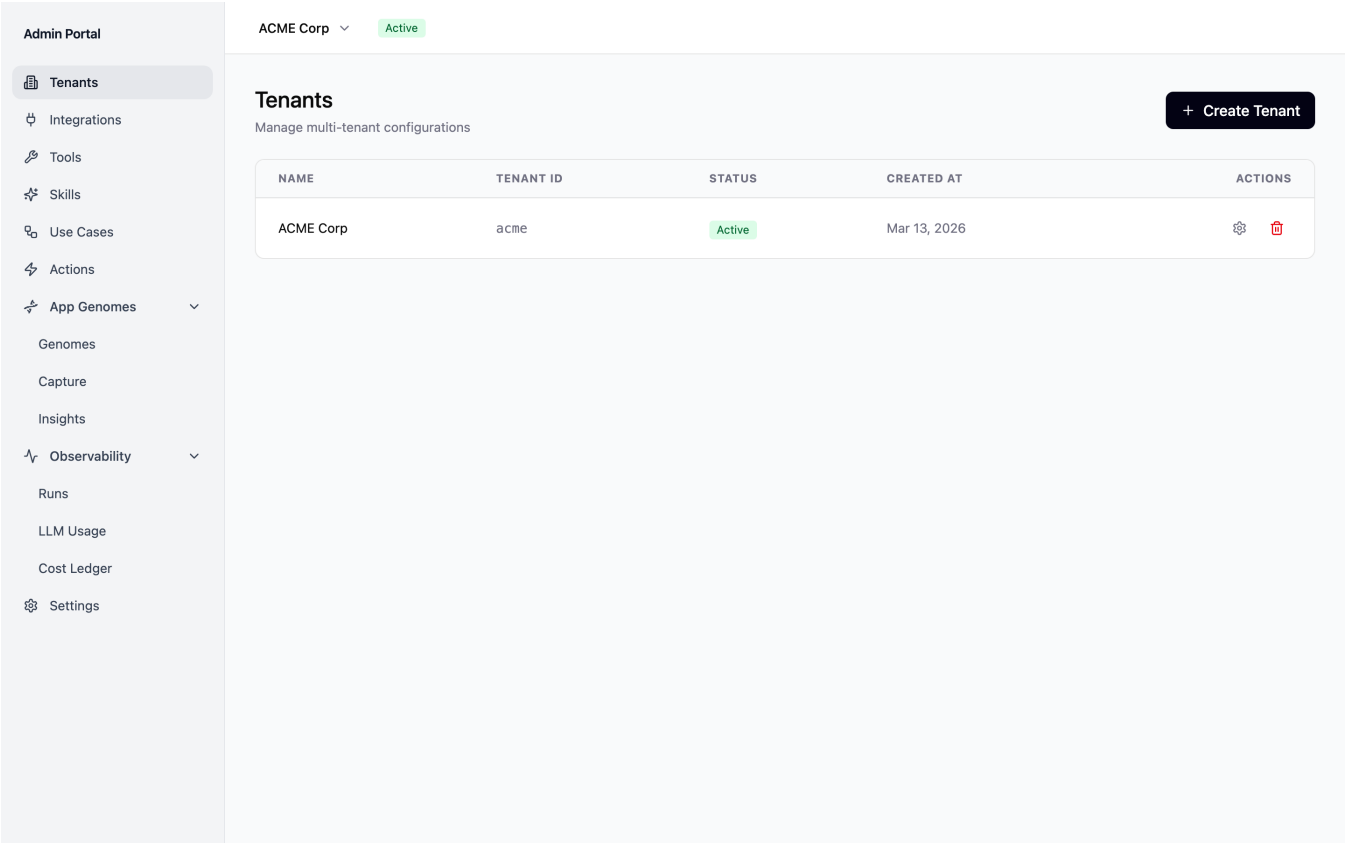
- **Tenants** — Multi-tenant isolation. Every resource is scoped to a tenant.
- **Integrations** — Connectivity to external systems (ServiceNow, GitHub, Jira, etc.) with auth credentials and webservice endpoints.
- **Tools** — Atomic capabilities backed by integration APIs. Each tool has typed input/output schemas.
- **Skills** — Reusable AI capabilities that compose one or more tools with instructions.
- **Use Cases** — Workflows that chain skills into multi-step sequences, matched to user intent via keyword triggers.
- **Actions** — Post-execution decisions that write back to external systems, recommended based on rules that score against the completed agent run.

Technology Stack

- **Backend**: Python, FastAPI, Pydantic models, in-memory store (ABC interface → MemoryStore)
- **Frontend**: React + TypeScript, Tailwind CSS, React Router, Sonner toasts
- **Pattern**: Repository pattern — ABC store interface → InMemory implementation → FastAPI router → app.state
- **Agent Stream**: Server-Sent Events (SSE) for real-time execution streaming

2. Module 1: Tenants

Tenants provide multi-tenant isolation. Every other module is scoped to a tenant_id. The platform supports an "All Tenants" global view for cross-tenant visibility.



Tenants list view — showing tenant name, ID, status, and created date

Data Model

Field	Type	Description
id	str	Unique identifier (e.g. 'acme')
name	str	Human-readable tenant name
status	str	'draft' or 'active'
shared_secret	Optional[str]	Shared secret for API auth
created_at	datetime	Creation timestamp
updated_at	datetime	Last update timestamp

API Endpoints

Method	Path	Description
POST	/api/tenants	Create a new tenant

GET	/api/tenants	List all tenants
GET	/api/tenants/{id}	Get single tenant
DELETE	/api/tenants/{id}	Delete tenant

Store Interface

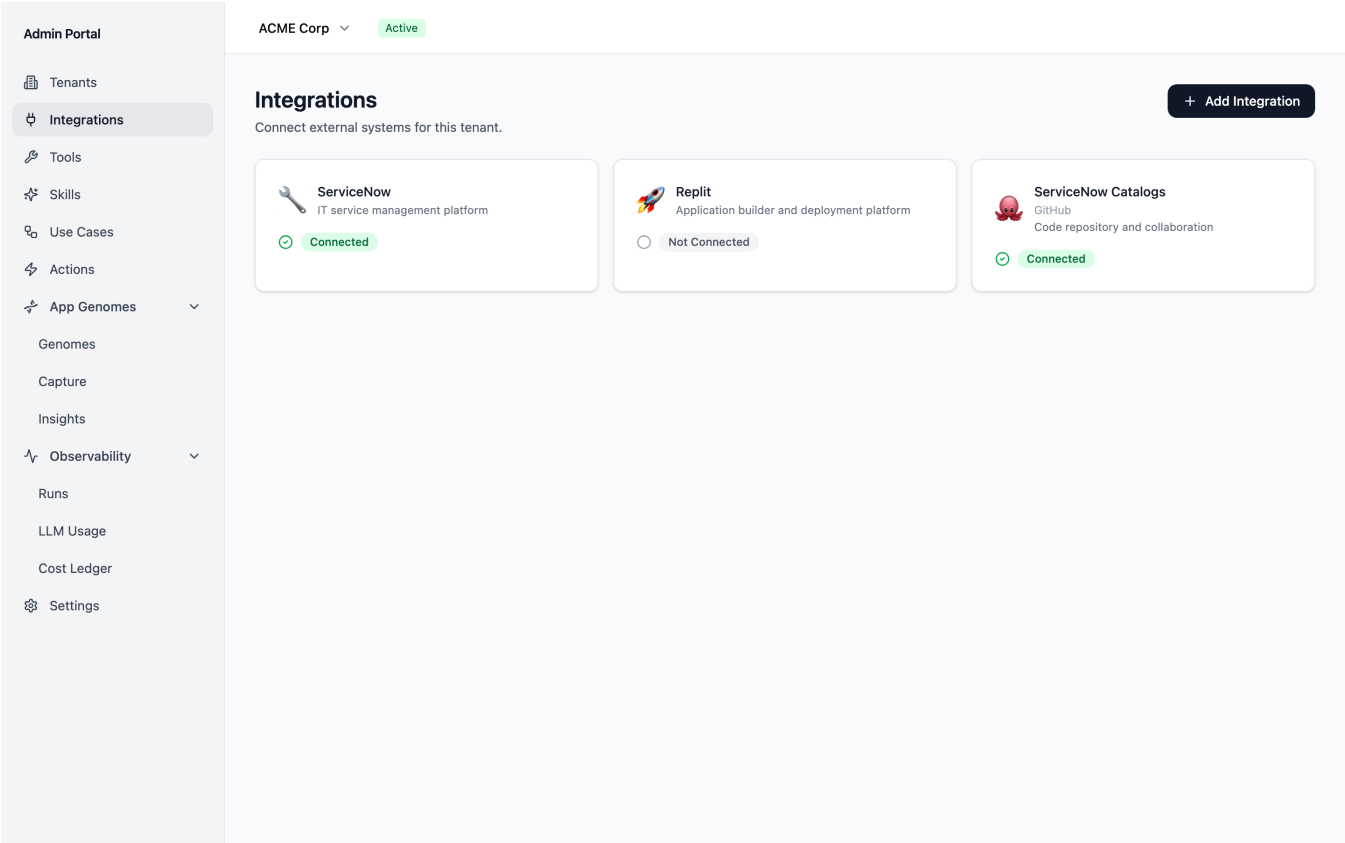
```
TenantStore: create(name) | get(id) | list() | delete(id) | update(id, **kwargs)
```

Frontend

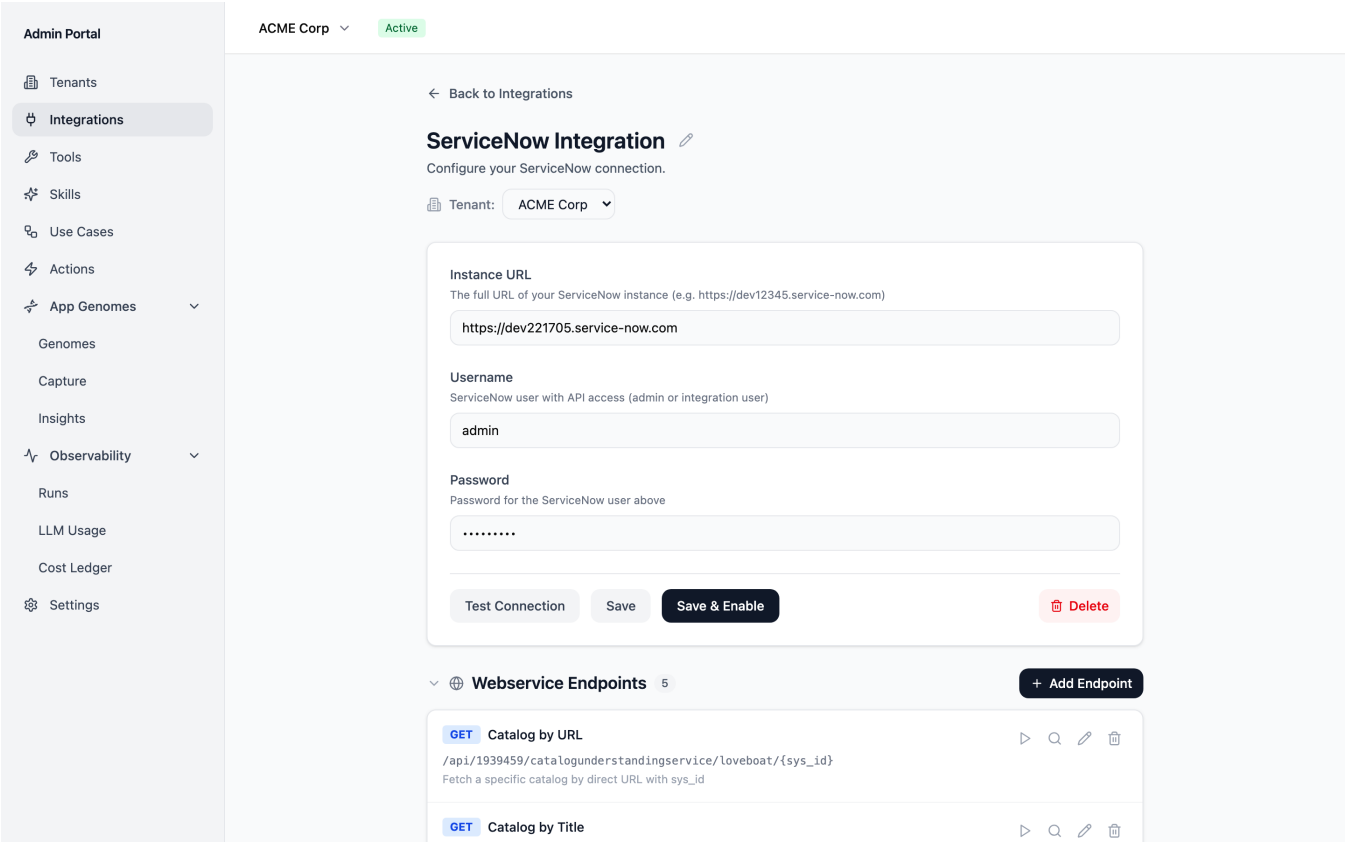
- **TenantsPage** — Table of all tenants with name, ID, status, created date, delete action
- **CreateTenantPage** — Multi-step wizard (Details → Integrations → Use Cases → Summary)
- **TenantContext** — Global React context; tracks currentTenantId, isAllTenants, apiTenantId; ALL_TENANTS = '__all__'
- **TopBar** — Tenant switcher dropdown with 'All Tenants' global view option

3. Module 2: Integrations

Integrations are the connectivity layer — connections to external systems with auth credentials and configurable webservice endpoints. Each integration belongs to a tenant and has a type from the catalog.



Integrations list — connectors configured per tenant with type, status, and endpoint count



Integration config detail — auth credentials, webservice endpoints, and live tester panel

Data Model: Integration

Field	Type	Description
id	str	Unique ID (int_*)
tenant_id	str	Owning tenant
integration_type	str	Type key (e.g. 'servicenow', 'github')
name	str	Custom display name
enabled	bool	Whether integration is active
config	dict	Auth credentials (instance_url, token, etc.)
endpoints	list[IntegrationEndpoint]	Configured webservice endpoints
created_at / updated_at	datetime	Timestamps

Data Model: IntegrationEndpoint

Field	Type	Description
id	str	Unique ID (ep_*)
name	str	Endpoint name (e.g. 'Search Incidents')
path	str	URL path (e.g. '/api/now/table/incident')

method	str	HTTP method (GET, POST, PUT, etc.)
headers	dict	Custom headers
query_params	dict	Default query parameters
description	str	What this endpoint does

Supported Integration Types

Type	Name	Config Fields	Default Endpoints
servicenow	ServiceNow	instance_url, username, password	6 (incidents, KB, catalogs)
google-drive	Google Drive	client_id, root_folder_id	0
salesforce	Salesforce	instance_url, username, password	3 (SOQL, create, describe)
slack	Slack	webhook_url	0
github	GitHub	token, org, repo	3 (repos, contents)
jira	Jira	instance_url, username, api_token	3 (search, create, get)
replit	Replit	connect_sid, username	0

API Endpoints (16 routes)

Method	Path	Description
GET	.../catalog	Return INTEGRATION_CATALOG
GET	.../	List integrations (supports filter_tenant)
POST	.../	Create integration (prevents duplicates except github)
GET	.../{id}	Get single integration
PUT	.../{id}/name	Rename integration
PUT	.../{id}/tenant	Reassign to different tenant
PUT	.../{id}/config	Update auth config (syncs to config stores)
PUT	.../{id}/enable | /disable	Toggle enabled flag
POST	.../{id}/test	Test connection (type-specific logic)
DELETE	.../{id}	Delete integration
POST	.../{id}/endpoints	Add webservice endpoint
PUT	.../{id}/endpoints/{eid}	Update endpoint
DELETE	.../{id}/endpoints/{eid}	Delete endpoint
POST	.../{id}/endpoints/{eid}/test	Test endpoint (with path_vars, limit)
POST	.../{id}/endpoints/{eid}/fetch	Fetch sample records (auto-parses response)

Key Features

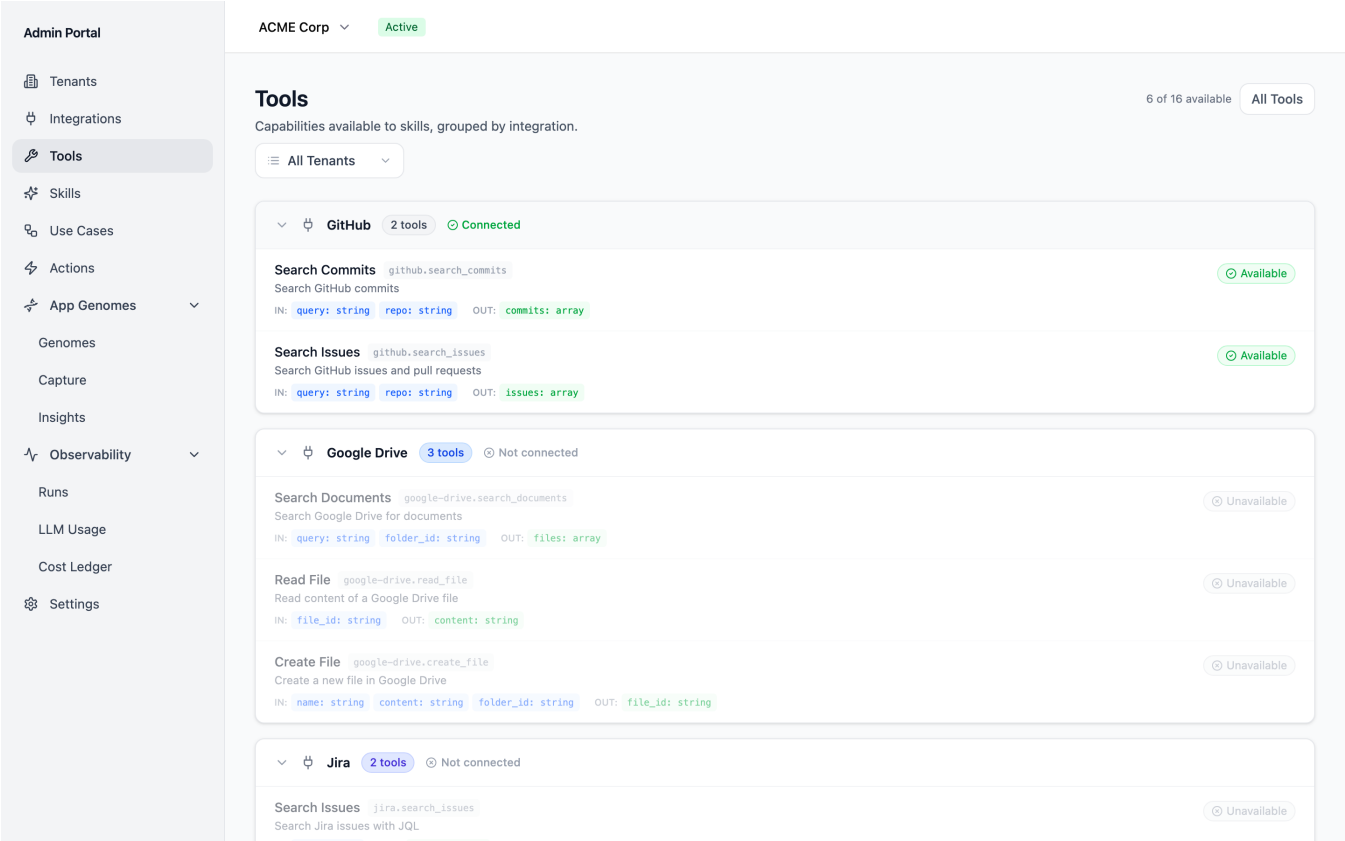
- **MULTI_INSTANCE_TYPES** — GitHub allows multiple integrations per tenant; others enforce one-per-type

• **Endpoint Tester** — Test + Fetch with path variable auto-detection ({var} prompt), limit control, response body viewer

• **Config Sync** — Saving config syncs to legacy ServiceNow/GoogleDrive/Replit config stores for backward compatibility

4. Module 3: Tools

Tools are atomic capabilities backed by integration APIs. They are defined in a static catalog (TOOL_CATALOG) and referenced by Skills. Each tool has typed input/output schemas and a handler function that executes the actual API call.



Tools catalog — grouped by integration type, showing availability based on enabled connectors

Tool Catalog (16 tools across 7 integrations)

tool_id	Integration	Name	Input	Output
servicenow.search_incidents	ServiceNow	Search Incidents	query, limit	incidents[]
servicenow.get_incident_details	ServiceNow	Get Incident Details	sys_id	incident
servicenow.search_kb	ServiceNow	Search Knowledge Base	query, limit	articles[]
servicenow.add_work_note	ServiceNow	Add Work Note	sys_id, note	ok
google-drive.search_documents	Google Drive	Search Documents	query, folder_id	files[]
google-drive.read_file	Google Drive	Read File	file_id	content
google-drive.create_file	Google Drive	Create File	name, content, folder_id	file_id
salesforce.search_accounts	Salesforce	Search Accounts	query	accounts[]
salesforce.get_case_history	Salesforce	Get Case History	account_id	cases[]

slack.send_message	Slack	Send Message	channel, text	ok, ts
slack.search_messages	Slack	Search Messages	query	messages[]
github.search_commits	GitHub	Search Commits	query, repo	commits[]
github.search_issues	GitHub	Search Issues	query, repo	issues[]
jira.search_issues	Jira	Search Issues	jql	issues[]
jira.get_issue	Jira	Get Issue	issue_key	issue
replit.build_application	Replit	Build Application	app_name, desc, stack	project_id, url

Tool Execution

The tool_executor.py service dispatches tool calls to integration-specific handlers:

```
execute_tool(tenant_id, tool_id, input_payload, app) → {status, tool_id, ...response}
```

- **_HANDLERS** maps tool_id → async handler function (e.g. servicenow_tools.search_incidents)
- Handler loads integration config from stores, makes HTTP request, returns structured result
- Status values: 'completed', 'not_implemented', 'error'

API Endpoints

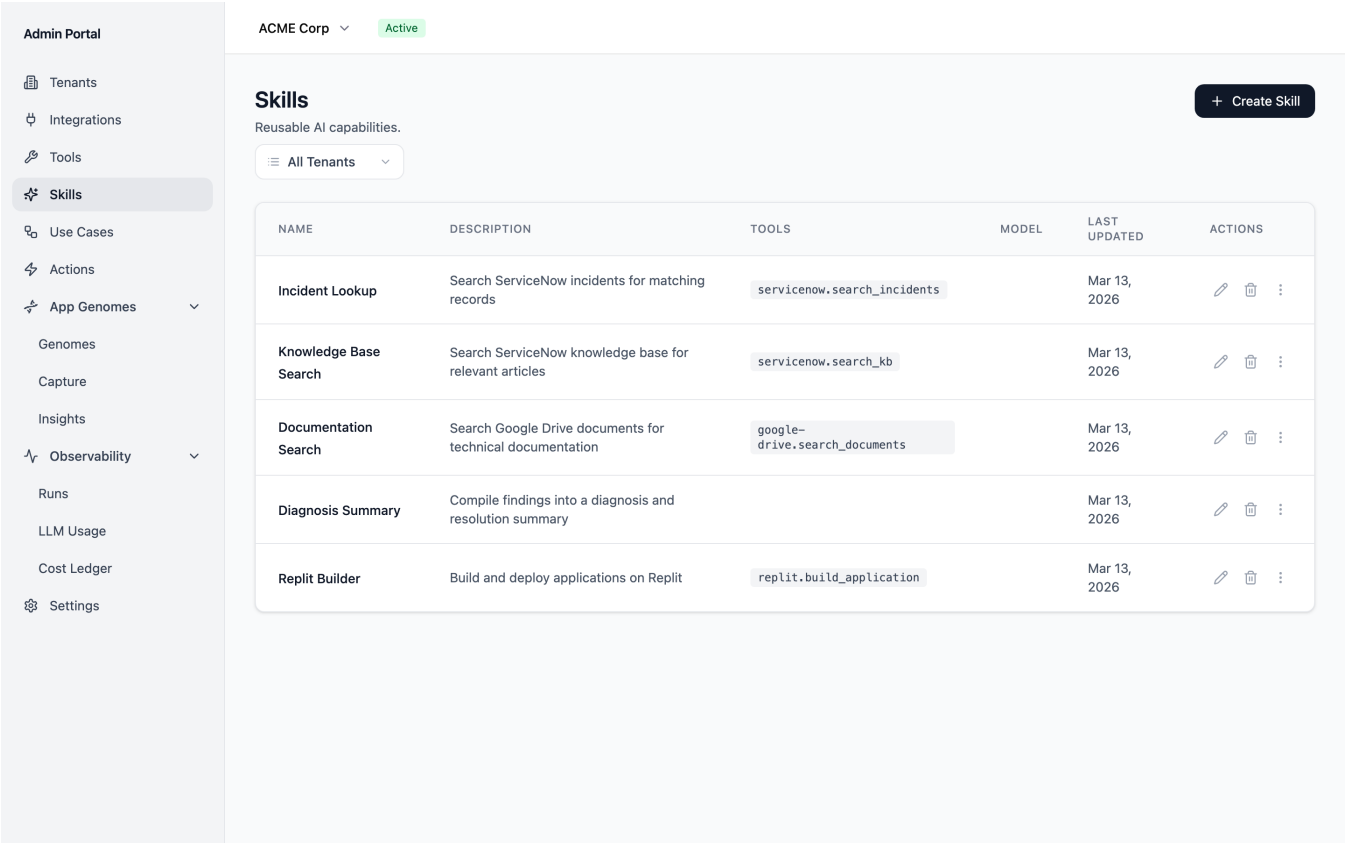
Method	Path	Description
GET	/api/admin/{tenant}/tools/catalog	All tools grouped by integration type
GET	/api/admin/{tenant}/tools/available	Only tools for enabled integrations

Availability Logic

A tool is 'available' only if its integration_type has an enabled integration with valid config for the tenant. This is computed at query time by checking IntegrationStore + config stores.

5. Module 4: Skills

Skills are reusable AI capabilities that compose one or more tools with LLM instructions. They are the building blocks that Use Cases chain together into workflows.



Skills catalog — each skill lists its assigned tools and LLM model

Data Model

Field	Type	Description
id	str	Unique ID (sk_*)
tenant_id	str	Owning tenant
name	str	Skill name (e.g. 'Incident Lookup')
description	str	What this skill does
model	str	LLM model to use (e.g. 'claude-sonnet-4-20250514')
instructions	str	Prompt instructions for the LLM
tools	list[str]	Array of tool_ids from TOOL_CATALOG
created_at / updated_at	datetime	Timestamps

API Endpoints

Method	Path	Description
GET	/api/admin/{tenant}/skills	List skills (supports filter_tenant)
POST	/api/admin/{tenant}/skills	Create skill (validates tool_ids)
GET	/api/admin/{tenant}/skills/{id}	Get single skill
PUT	/api/admin/{tenant}/skills/{id}	Update skill (re-validates tools)
DELETE	/api/admin/{tenant}/skills/{id}	Delete skill

Validation

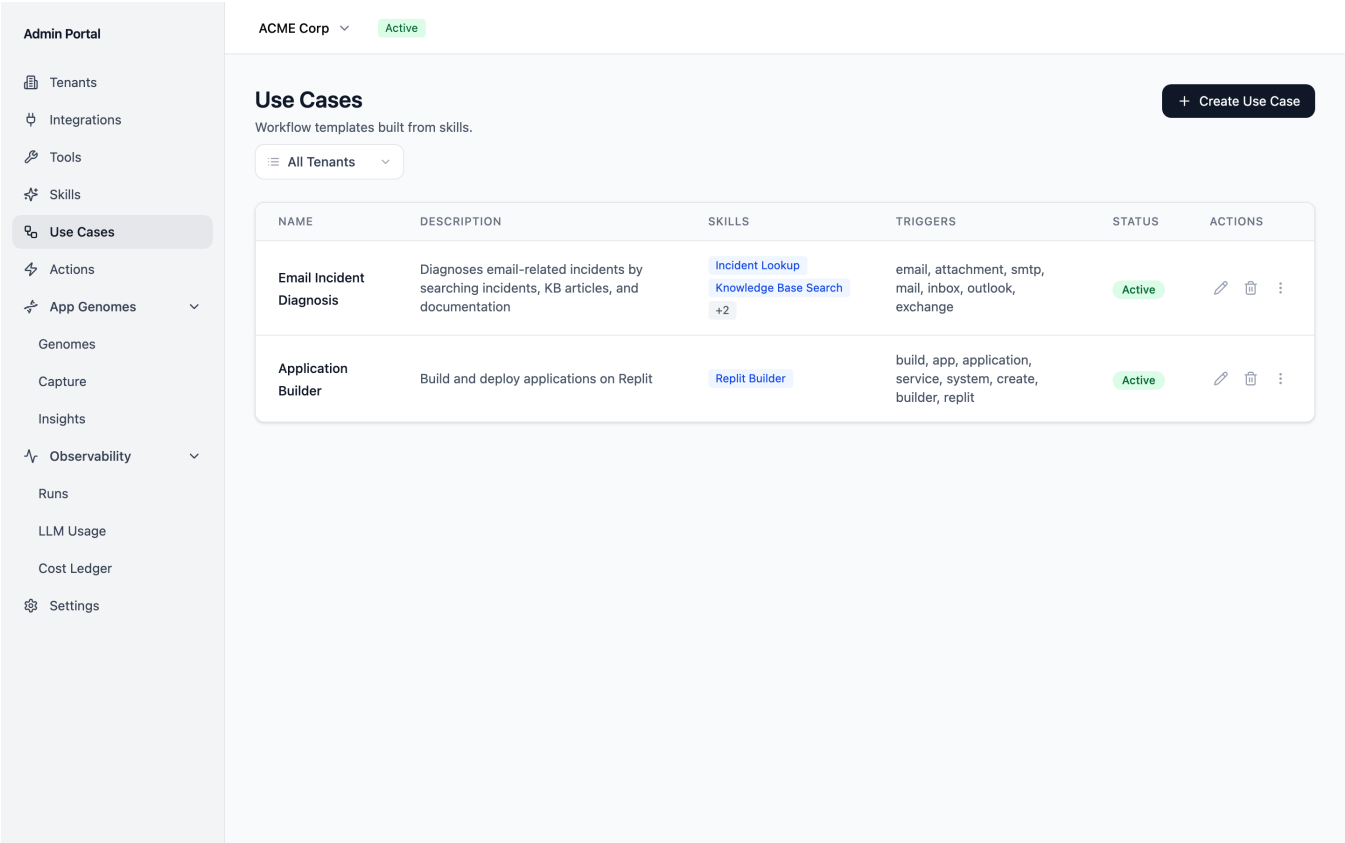
On create and update, all tool_ids in the tools[] array are validated against TOOL_CATALOG_BY_ID. Unknown tool_ids cause a 400 error listing the invalid IDs.

Seed Skills (ACME tenant)

Skill Name	Tools	Purpose
Incident Lookup	servicenow.search_incidents	Search ServiceNow incident records
Knowledge Base Search	servicenow.search_kb	Search ServiceNow KB articles
Documentation Search	google-drive.search_documents	Search Google Drive for docs
Diagnosis Summary	(none)	LLM-only: synthesize findings
Replit Builder	replit.build_application	Build an app on Replit

6. Module 5: Use Cases

Use Cases are workflows that chain Skills into multi-step sequences. They are matched to user prompts via keyword triggers and executed by the agent orchestration layer.



Use Cases — workflows with trigger keywords and ordered skill steps

Data Model: UseCase

Field	Type	Description
id	str	Unique ID (uc_*)
tenant_id	str	Owning tenant
name	str	Use case name
description	str	What this workflow does
status	'draft' | 'active'	Only active use cases are matched by agent
triggers	list[str]	Keywords for matching (e.g. ['email','smtp','outlook'])
steps	list[UseCaseStep]	Ordered sequence of skill executions
created_at / updated_at	datetime	Timestamps

Data Model: UseCaseStep

Field	Type	Description
step_id	str	Unique step ID
skill_id	str	Reference to Skill.id
name	str	Step display name
input_mapping	str	How to map input from prior steps
output_mapping	str	How to expose output to next steps

Use Case Matching (Agent Stream)

When a user sends a prompt to the agent stream endpoint (/api/admin/{tenant}/agent/stream):

- 1. Load all **active** use cases for the tenant
- 2. Score each use case against the prompt using **_score_use_case()** — keyword overlap between prompt tokens and use case triggers
- 3. Select the best-scoring use case (if score >= 0.05)
- 4. Execute each step sequentially: load skill → execute tools → capture results
- 5. Stream SSE events to frontend: reasoning, use_case_selected, skill_started, tool_called, tool_result, skill_completed, final_result
- 6. Persist AgentUIRun with selected_use_case, skills_used, result, confidence

API Endpoints

Method	Path	Description
GET	.../use-cases	List use cases (supports filter_tenant)
POST	.../use-cases	Create (validates step skill_ids exist)
GET	.../use-cases/{id}	Get single use case
PUT	.../use-cases/{id}	Update (re-validates steps)
DELETE	.../use-cases/{id}	Delete use case
POST	.../use-cases/{id}/run	Execute use case (background task)
GET	.../use-cases/{id}/runs	List all runs for a use case
GET	.../use-cases/{id}/runs/{rid}	Get single run detail
GET	.../use-cases/{id}/runs/{rid}/events	SSE stream of run events

Seed Use Cases (ACME)

Email Incident Diagnosis (active) — triggers: email, attachment, smtp, mail, inbox, outlook, exchange

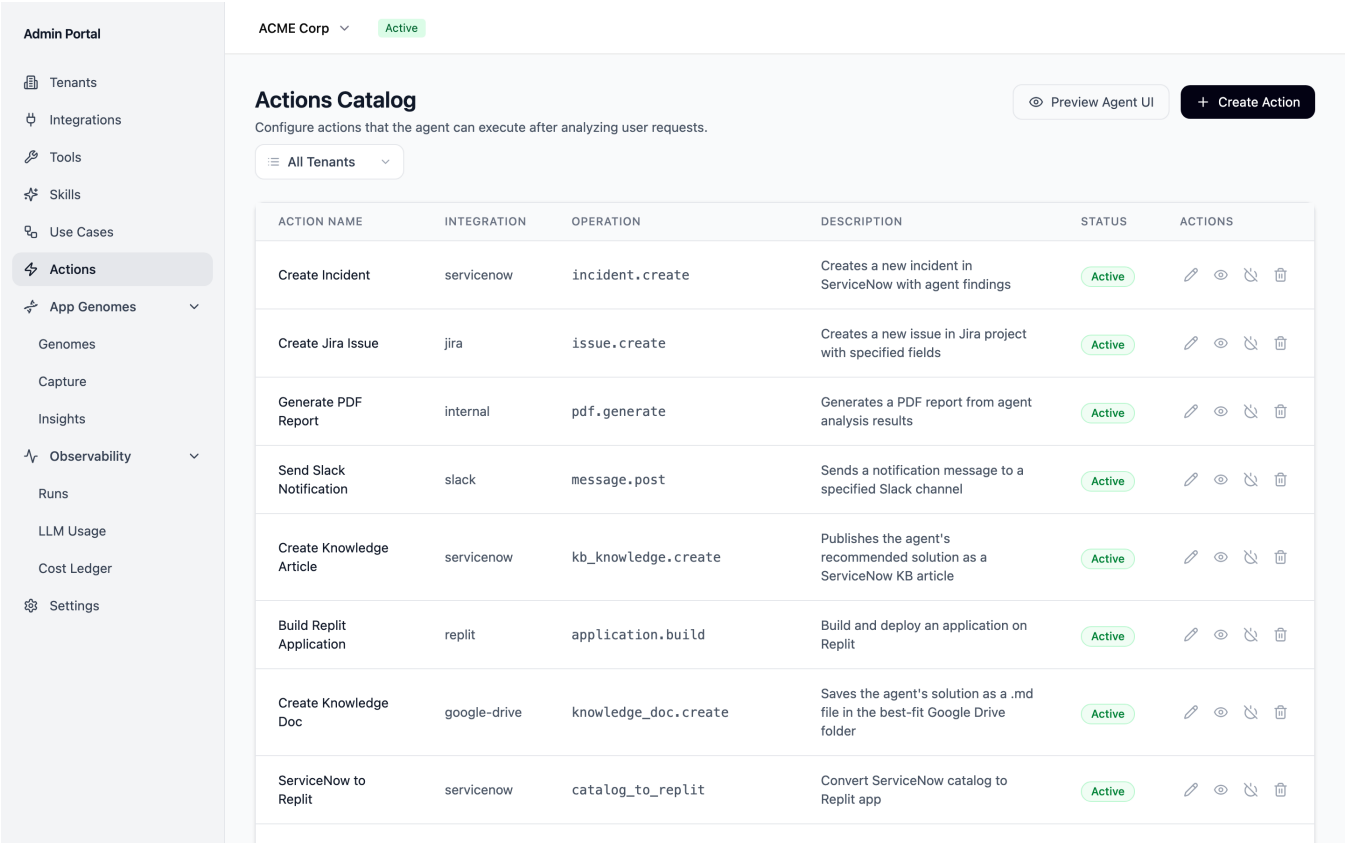
- Step 1: Incident Lookup (servicenow.search_incidents)
- Step 2: Knowledge Base Search (servicenow.search_kb)
- Step 3: Documentation Search (google-drive.search_documents)
- Step 4: Diagnosis Summary (LLM synthesis, no tools)

Application Builder (active) — triggers: build, app, application, service, system, create, builder, replit

- Step 1: Replit Builder (replit.build_application)

7. Module 6: Actions

Actions are post-execution decisions that write back to external systems. After the agent completes a use case run, Actions are scored and recommended based on configurable rules.



Actions catalog — post-execution writebacks with integration targets and recommendation rules

Data Model: Action

Field	Type	Description
id	str	Unique ID (act_*)
tenant_id	str	Owning tenant
name	str	Action name (e.g. 'Create Incident')
description	str	What this action does
integration_id	str	Target system (e.g. 'servicenow', 'jira')
operation	str	Operation type (e.g. 'incident.create')
parameters	list[ActionParameter]	How to resolve inputs
rules	list[ActionRule]	When to recommend this action
status	'active' | 'disabled'	Whether action is available

ActionParameter

Field	Type	Values
name	str	Parameter name (e.g. 'title', 'description')
source	str	static | user_prompt | agent_result | user_input | agent_metadata
value	Optional[str]	Static value or default

Parameter Resolution: static → use param.value; user_prompt → run.prompt; agent_result → run.result; user_input → ask user via UI; agent_metadata → reserved

ActionRule

Field	Type	Values
type	str	use_case | confidence | skill | keyword | tag
operator	str	equals | not_equals | greater_than | less_than | contains | not_contains
value	str	Value to match against (e.g. use case name, keyword list, threshold)

Recommendation Scoring

After a completed agent run, the recommendation engine scores each active action:

Rule Type	Points	Logic
use_case	+3	run.selected_use_case matches rule value (case-insensitive)
keyword	+2	Comma-separated keywords overlap with prompt tokens
skill	+2	Comma-separated skill names overlap with run.skills_used
confidence	+1	run.confidence satisfies operator against threshold value

- **Score >= 2** → 'recommended' list (shown prominently in UI)
- **Score < 2** → 'available' list (shown as secondary options)

Operation Handlers

integration:operation	Handler
servicenow:incident.create	servicenow_tools.create_incident
servicenow:kb_knowledge.create	servicenow_tools.create_knowledge_article
servicenow:catalog_to_replit	snow_to_replit.convert_catalog_to_replit
servicenow:catalog_to_github	snow_to_github.convert_catalog_to_github
jira:issue.create	jira_tools.create_issue
slack:message.post	slack_tools.post_message
google-drive:knowledge_doc.create	google_drive_tools.create_knowledge_doc
replit:application.build	replit_tools.build_application_action
internal:pdf.generate	pdf_tools.generate_pdf

API Endpoints

Method	Path	Description
GET	.../actions	List actions (supports filter_tenant)
POST	.../actions	Create action
GET	.../actions/{id}	Get single action
PUT	.../actions/{id}	Update action
DELETE	.../actions/{id}	Delete action
POST	.../actions/{id}/execute	Execute action (resolve params, dispatch)
GET	.../actions/recommendations/{run_id}	Score + return recommended/available

8. Cross-Module Relationships

Dependency Chain

```
Tenant → Integration → Tool → Skill → UseCase → Action
```

From	To	Relationship	How
Tenant	All modules	Scoping	Every resource has tenant_id; TenantContext drives UI filtering
Integration	Tools	Availability	Tools filtered by enabled integrations; /tools/available checks IntegrationStore
Integration	Actions	Execution	Action handlers use integration config for API calls
Tools	Skills	Composition	Skill.tools[] references tool_ids from TOOL_CATALOG
Skills	Use Cases	Workflow steps	UseCaseStep.skill_id references Skill.id
Use Cases	Agent Stream	Selection	Agent scores use cases against prompt, selects best match
Agent Run	Actions	Recommendations	Action rules scored against completed AgentUIRun attributes
Integration	Endpoints	Embedded	IntegrationEndpoint[] stored directly on Integration model

Data Flow

- **Configuration time:** Admin creates Tenant → configures Integrations → sees available Tools → builds Skills → composes Use Cases → defines Actions with rules
- **Runtime:** User prompt → Use Case matched → Skills executed (tools called) → AgentUIRun persisted → Actions recommended → User approves → Action executed against integration

9. End-to-End Execution Flow

Example: User types "I have an email outage affecting our team"

Phase 1: Use Case Matching

- 1. Frontend calls POST /api/admin/{tenant}/agent/stream with prompt
- 2. Backend loads all **active** use cases for tenant
- 3. _score_use_case() scores each: 'Email Incident Diagnosis' matches on ['email'] → score 0.75
- 4. Emit SSE: use_case_selected {name, confidence: 0.75}

Phase 2: Skill Execution

- 5. Step 1 — **Incident Lookup**: execute servicenow.search_incidents → returns incident records
- 6. Step 2 — **Knowledge Base Search**: execute servicenow.search_kb → returns KB articles
- 7. Step 3 — **Documentation Search**: execute google-drive.search_documents → returns docs
- 8. Step 4 — **Diagnosis Summary**: LLM synthesizes all findings
- 9. Each step emits SSE events: skill_started, tool_called, tool_result, skill_completed

Phase 3: Action Recommendation

- 10. AgentUIRun persisted with selected_use_case, skills_used, result, confidence
- 11. Frontend calls GET /actions/recommendations/{run_id}
- 12. 'Create Incident' scores: +3 (use_case) +2 (keywords) +2 (skill) +1 (confidence) = **8 points** → **recommended**
- 13. 'Create Jira Issue' scores: +2 (keywords) = 2 points → recommended
- 14. 'Send Slack Notification' scores: +2 (keywords) = 2 points → recommended

Phase 4: Action Execution

- 15. User clicks 'Create Incident'
- 16. Parameter resolution: title ← run.prompt, description ← run.result, priority ← '3' (static)
- 17. Dispatched to servicenow_tools.create_incident(tenant_id, params, app)
- 18. ServiceNow API call: POST /api/now/table/incident with auth from integration config
- 19. Result: 'Incident INC0123456 created' displayed in chat

10. Data Model Summary

Quick reference of all core models and their key fields:

Model	Key Fields	Store	Relationships
Tenant	id, name, status	TenantStore	Scopes all other models
Integration	id, tenant_id, type, config, endpoints[]	IntegrationStore	Determines tool availability
IntegrationEndpoint	id, name, path, method	(embedded)	Belongs to Integration
Tool (catalog)	tool_id, integration_type, schemas	(static dict)	Referenced by Skill.tools[]
Skill	id, tenant_id, name, tools[], instructions	SkillStore	Referenced by UseCaseStep
UseCase	id, tenant_id, triggers[], steps[]	UseCaseStore	Matched by agent stream
UseCaseStep	step_id, skill_id	(embedded)	Belongs to UseCase
Action	id, tenant_id, operation, params[], rules[]	ActionStore	Scored against AgentUIRun
ActionParameter	name, source, value	(embedded)	Resolved at execution time
ActionRule	type, operator, value	(embedded)	Drives recommendation scoring
AgentUIRun	run_id, prompt, selected_use_case, result	AgentUIRunStore	Links use case → actions

End of Document

Generated for OverYonder.ai — March 2026